FRD

Functional Requirements Document (FRD)

AWS Glue ETL Pipeline for Customer Order Processing

•

1. Document Overview

Purpose

This Functional Requirements Document defines the functional behavior of an AWS Glue-based ETL pipeline that ingests customer and order data from S3, performs data quality operations, implements Slowly Changing Dimension Type 2 (SCD2) logic using Apache Hudi, and produces analytical aggregations for business intelligence purposes.

Scope

This FRD covers:

- Ingestion of customer and order CSV files from S3
- Data cleaning and deduplication
- Registration of datasets in AWS Glue Data Catalog
- Implementation of SCD Type 2 for order summary tracking
- Incremental processing triggered by customer data changes
- Generation of customer aggregate spend analytics
- Use of AWS services: S3, Glue, Hudi, Athena, CloudWatch, and optionally Lake Formation

This FRD does NOT cover:

- Upstream data generation or validation
- Downstream BI tool integration
- User interface or reporting layer
- Data retention and archival policies
- Disaster recovery procedures

Assumptions

1. CSV files in the source S3 paths are well-formed and use standard delimiters
2. CustId in both datasets is the join key and is assumed to be consistent
3. "Null" (string) and NULL (actual null) are the only null representations requiring removal
4. Duplicate detection is based on exact row matching across all columns
5. Date field in Order dataset is in a parseable format (specific format not provided in BRD)

6. SCD2 change detection is based on any change in customer or order attributes

7. OpTs (operation timestamp) will be system-generated at processing time

8. AWS Glue job has appropriate IAM permissions for S3, Glue Catalog, and CloudWatch

9. Hudi table format is compatible with Athena for querying

10. Incremental processing assumes availability of change data or full refresh with change detection logic

•

2. Functional Requirements

FR-INGEST-001: Ingest Customer Data from S3

• Title: Load Customer CSV Files from S3

• Description:

The system shall read customer data in CSV format from the specified S3 location and load it into the processing pipeline.

• Business Objective:

Enable the ETL pipeline to access current customer information for downstream processing and analytics.

• Inputs:

• Source: s3://adif-sdlc/sdlc_wizard/customerdata/

• Format: CSV

• Schema:

• CustId: (datatype not specified in BRD)

• Name: (datatype not specified in BRD)

• EmailId: (datatype not specified in BRD)

• Region: (datatype not specified in BRD)

• Processing Rules / Functional Steps:

1. Connect to S3 path s3://adif-sdlc/sdlc_wizard/customerdata/

2. Read all CSV files present in the location

3. Parse CSV content using standard CSV parsing logic

4. Load data into a DataFrame or equivalent processing structure

• Outputs:

• In-memory customer dataset ready for cleaning

• Preconditions:

• S3 bucket and path exist and are accessible

• AWS Glue job has read permissions on the S3 path

• At least one CSV file exists in the source path

• Postconditions:

• Customer data is loaded into memory for further processing

• No data transformation has occurred at this stage

• Error / Validation Conditions:

• If S3 path does not exist, job must fail with clear error message

• If no CSV files are found, job must fail or log warning based on configuration

• If CSV parsing fails, job must log the error and fail

•

FR-INGEST-002: Ingest Order Data from S3

• Title: Load Order CSV Files from S3

• Description:

The system shall read order data in CSV format from the specified S3 location and load it into the processing pipeline.

• Business Objective:

Enable the ETL pipeline to access current order transaction information for downstream processing and analytics.

• Inputs:

• Source: s3://adif-sdlc/sdlc_wizard/orderdata/

• Format: CSV

• Schema:

• OrderId: (datatype not specified in BRD)

• ItemName: (datatype not specified in BRD)

• PricePerUnit: (datatype not specified in BRD)

• Qty: (datatype not specified in BRD)

• Date: (datatype not specified in BRD)

• CustId: (datatype not specified in BRD)

• Processing Rules / Functional Steps:

1. Connect to S3 path s3://adif-sdlc/sdlc_wizard/orderdata/

2. Read all CSV files present in the location

3. Parse CSV content using standard CSV parsing logic

4. Load data into a DataFrame or equivalent processing structure

• Outputs:

• In-memory order dataset ready for cleaning

• Preconditions:

• S3 bucket and path exist and are accessible

• AWS Glue job has read permissions on the S3 path

• At least one CSV file exists in the source path

• Postconditions:

• Order data is loaded into memory for further processing

• No data transformation has occurred at this stage

• Error / Validation Conditions:

• If S3 path does not exist, job must fail with clear error message

• If no CSV files are found, job must fail or log warning based on configuration

• If CSV parsing fails, job must log the error and fail

•

FR-CLEAN-001: Remove Null Values from Customer and Order Datasets

• Title: Data Quality - Null Value Removal

• Description:

The system shall remove all records containing "Null" (string literal) or NULL (actual null value) from both customer and order datasets.

• Business Objective:

Ensure data quality by eliminating incomplete records that could cause downstream processing errors or inaccurate analytics.

• Inputs:

• Raw customer dataset from FR-INGEST-001

• Raw order dataset from FR-INGEST-002

• Processing Rules / Functional Steps:

1. For customer dataset:

• Identify rows where any column contains the string "Null" (case-sensitive)

• Identify rows where any column contains actual NULL value

• Remove all identified rows

2. For order dataset:

• Identify rows where any column contains the string "Null" (case-sensitive)

• Identify rows where any column contains actual NULL value

• Remove all identified rows

3. Log count of removed records for each dataset

• Outputs:

• Cleaned customer dataset with no null values

• Cleaned order dataset with no null values

• Log entry with count of removed records

• Preconditions:

• Customer and order datasets have been successfully loaded

• Postconditions:

• All remaining records have complete data in all columns

• Record counts are reduced by the number of null-containing rows

• Error / Validation Conditions:

• If all records are removed, log warning but continue processing

• Processing should not fail if no null values are found

•

FR-CLEAN-002: Remove Duplicate Records from Customer and Order Datasets

• Title: Data Quality - Duplicate Removal

• Description:

The system shall remove duplicate records from both customer and order datasets based on exact row matching across all columns.

• Business Objective:

Prevent duplicate processing and ensure accurate aggregations by maintaining unique records only.

• Inputs:

• Null-cleaned customer dataset from FR-CLEAN-001

• Null-cleaned order dataset from FR-CLEAN-001

• Processing Rules / Functional Steps:

1. For customer dataset:

• Compare all rows based on all column values

• Identify exact duplicates

• Retain only one instance of each unique row

• Remove all duplicate instances

2. For order dataset:

• Compare all rows based on all column values

• Identify exact duplicates

• Retain only one instance of each unique row

• Remove all duplicate instances

3. Log count of removed duplicates for each dataset

• Outputs:

• Deduplicated customer dataset

• Deduplicated order dataset

• Log entry with count of removed duplicates

• Preconditions:

• Null removal (FR-CLEAN-001) has been completed

• Postconditions:

• Each unique record appears exactly once in each dataset

• Record counts are reduced by the number of duplicate rows

• Error / Validation Conditions:

• Processing should not fail if no duplicates are found

• If all records are duplicates (leaving zero records), log error and fail

•

FR-CATALOG-001: Register Customer Table in Glue Data Catalog

• Title: Catalog Customer Data as Glue Table

• Description:

The system shall register the cleaned customer dataset as a table in the AWS Glue Data Catalog for query access and metadata management.

• Business Objective:

Enable SQL-based querying of customer data through Athena and maintain centralized metadata for data governance.

• Inputs:

• Deduplicated customer dataset from FR-CLEAN-002

• Processing Rules / Functional Steps:

1. Write cleaned customer data to S3 in a queryable format (Parquet or similar)

2. Create or update Glue Catalog table with the following properties:

• Database: gen_ai_poc_databrickscoe

• Table name: sdlc_wizard_customer

• Schema: CustId, Name, EmailId, Region

• Location: S3 path where customer data is written

3. Update table metadata and statistics

• Outputs:

• Glue Catalog table: gen_ai_poc_databrickscoe.sdlc_wizard_customer

• Customer data persisted in S3 in cataloged location

• Preconditions:

• Glue database gen_ai_poc_databrickscoe exists

• AWS Glue job has permissions to create/update catalog tables

• Customer data cleaning is complete

• Postconditions:

• Customer table is queryable via Athena

• Table metadata is current and accurate

• Error / Validation Conditions:

• If database does not exist, job must fail with clear error

• If table creation fails, job must fail and roll back S3 writes if possible

• If S3 write fails, job must not create catalog entry

•

FR-CATALOG-002: Register Order Table in Glue Data Catalog

• Title: Catalog Order Data as Glue Table

• Description:

The system shall register the cleaned order dataset as a table in the AWS Glue Data Catalog for query access and metadata management.

• Business Objective:

Enable SQL-based querying of order data through Athena and maintain centralized metadata for data governance.

• Inputs:

• Deduplicated order dataset from FR-CLEAN-002

• Processing Rules / Functional Steps:

1. Write cleaned order data to S3 in a queryable format (Parquet or similar)

2. Create or update Glue Catalog table with the following properties:

• Database: gen_ai_poc_databrickscoe

• Table name: sdlc_wizard_order

• Schema: OrderId, ItemName, PricePerUnit, Qty, Date, CustId

• Location: S3 path where order data is written

3. Update table metadata and statistics

• Outputs:

• Glue Catalog table: gen_ai_poc_databrickscoe.sdlc_wizard_order

• Order data persisted in S3 in cataloged location

• Preconditions:

• Glue database gen_ai_poc_databrickscoe exists

• AWS Glue job has permissions to create/update catalog tables

• Order data cleaning is complete

• Postconditions:

• Order table is queryable via Athena

• Table metadata is current and accurate

• Error / Validation Conditions:

• If database does not exist, job must fail with clear error

• If table creation fails, job must fail and roll back S3 writes if possible

• If S3 write fails, job must not create catalog entry

•

FR-SCD2-001: Implement SCD Type 2 for Order Summary

• Title: Slowly Changing Dimension Type 2 Processing for Order Summary

• Description:

The system shall join customer and order data, detect changes, and maintain historical versions of order summary records using SCD Type 2 methodology with Apache Hudi format on S3.

• Business Objective:

Maintain complete historical tracking of order and customer data changes to support temporal analysis and audit requirements.

• Inputs:

• Cleaned customer dataset from FR-CLEAN-002

• Cleaned order dataset from FR-CLEAN-002

• Existing ordersummary Hudi table (if present)

• Processing Rules / Functional Steps:

1. Join customer and order datasets on CustId

2. For each joined record, compare against existing active records in ordersummary table

3. If record is new (no match on business key):

• Insert new record with IsActive = true, StartDate = current date, EndDate = null, OpTs = current timestamp

4. If record exists and has changed (any attribute differs):

• Update existing record: set IsActive = false, EndDate = current date

• Insert new record with IsActive = true, StartDate = current date, EndDate = null, OpTs = current timestamp

5. If record exists and has not changed:

• No action required

6. Write results to Hudi table at s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

7. Register or update ordersummary table in Glue Catalog database gen_ai_poc_databrickscoe

• Outputs:

• Hudi table: ordersummary at s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

• Glue Catalog table: gen_ai_poc_databrickscoe.ordersummary

• Records with SCD2 attributes: IsActive, StartDate, EndDate, OpTs

• Preconditions:

• Customer and order data have been cleaned and are available

• S3 target location is accessible

• Hudi libraries are available in Glue environment

• Glue database gen_ai_poc_databrickscoe exists

• Postconditions:

• All current records have IsActive = true and EndDate = null

• All superseded records have IsActive = false and EndDate populated

• Complete history of changes is preserved

• Table is queryable via Athena with Hudi connector

• Error / Validation Conditions:

• If join produces no records, log warning and create empty table

• If Hudi write fails, job must fail and not update catalog

• If customer or order data is missing CustId, those records must be excluded from join

• If StartDate or EndDate cannot be determined, use system date as fallback

•

FR-INCR-001: Incremental Processing Based on Customer Updates

• Title: Trigger SCD2 Updates on Customer Data Changes

• Description:

The system shall detect changes in customer data and trigger corresponding SCD Type 2 updates in the ordersummary table to maintain consistency.

• Business Objective:

Ensure that changes to customer master data are reflected in the order summary historical records without requiring full reprocessing.

• Inputs:

• New or updated customer records from s3://adif-sdlc/sdlc_wizard/customerdata/

• Existing ordersummary Hudi table

• Processing Rules / Functional Steps:

1. Identify customer records that are new or have changed since last processing

2. Retrieve all active order summary records associated with changed customers (match on CustId)

3. For each affected order summary record:

• Close current active record: set IsActive = false, EndDate = current date

• Create new record with updated customer information: set IsActive = true, StartDate = current date, EndDate = null, OpTs = current timestamp

4. Write updates to ordersummary Hudi table using upsert operation

5. Maintain IsActive, StartDate, EndDate, OpTs attributes consistently

• Outputs:

• Updated ordersummary Hudi table with new versions for affected records

• Log of processed customer changes and affected order summary records

• Preconditions:

• Customer data ingestion (FR-INGEST-001) has completed

• ordersummary table exists and is accessible

• Change detection mechanism is available (timestamp, version, or full comparison)

• Postconditions:

• All order summary records linked to changed customers have new active versions

• Previous versions are properly closed with EndDate populated

• Historical integrity is maintained

• Error / Validation Conditions:

• If no customer changes are detected, skip processing and log informational message

• If ordersummary table is not found, log error and fail

• If Hudi upsert fails, job must fail and not commit partial updates

•

FR-AGG-001: Generate Customer Aggregate Spend Table

• Title: Create Customer Aggregate Spend Analytics Table

• Description:

The system shall aggregate order data by customer to calculate total spending and produce an analytics table for business intelligence purposes.

• Business Objective:

Provide summarized customer spending metrics to support sales analysis, customer segmentation, and revenue reporting.

• Inputs:

• Cleaned customer dataset from FR-CLEAN-002

• Cleaned order dataset from FR-CLEAN-002

• Processing Rules / Functional Steps:

1. Join customer and order datasets on CustId

2. Calculate TotalAmount for each order: TotalAmount = PricePerUnit Qty

3. Group by customer Name and Date

4. Aggregate: sum(TotalAmount) as TotalAmount

5. Select columns: Name, TotalAmount, Date

6. Write results to S3 location: s3://adif-sdlc/analytics/customeraggregatespend/

7. Register or update customeraggregatespend table in Glue Catalog database gen_ai_poc_databrickscoe

• Outputs:

• Analytics table at s3://adif-sdlc/analytics/customeraggregatespend/

• Glue Catalog table: gen_ai_poc_databrickscoe.customeraggregatespend

• Schema: Name, TotalAmount, Date

• Preconditions:

• Customer and order data cleaning is complete

• S3 target location is accessible

• Glue database gen_ai_poc_databrickscoe exists

• Postconditions:

• Aggregate spend data is available for querying via Athena

• One record per customer per date with total spending

• Table metadata is registered in Glue Catalog

• Error / Validation Conditions:

• If join produces no records, create empty table and log warning

• If PricePerUnit or Qty are non-numeric, exclude those records and log error

• If aggregation fails, job must fail with clear error message

• If S3 write fails, job must not create catalog entry

•

FR-CATALOG-003: Register Order Summary Table in Glue Data Catalog

• Title: Catalog Order Summary Hudi Table

• Description:

The system shall register the ordersummary Hudi table in the AWS Glue Data Catalog to enable Athena querying with Hudi format support.

• Business Objective:

Enable SQL-based historical analysis of order summary data through Athena while maintaining SCD2 temporal attributes.

• Inputs:

• ordersummary Hudi table at s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

• Processing Rules / Functional Steps:

1. Create or update Glue Catalog table with the following properties:

• Database: gen_ai_poc_databrickscoe

• Table name: ordersummary

• Location: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

• Format: Hudi

• Schema: All columns from customer and order join plus IsActive, StartDate, EndDate, OpTs

2. Configure Hudi-specific table properties for Athena compatibility

3. Update table metadata and statistics

• Outputs:

• Glue Catalog table: gen_ai_poc_databrickscoe.ordersummary

• Table queryable via Athena with Hudi connector

• Preconditions:

• ordersummary Hudi table has been created (FR-SCD2-001)

• Glue database gen_ai_poc_databrickscoe exists

• Athena workgroup supports Hudi format

• Postconditions:

• ordersummary table is queryable via Athena

• SCD2 attributes are accessible in queries

• Table metadata reflects Hudi format

• Error / Validation Conditions:

• If Hudi table does not exist, job must fail

• If catalog registration fails, log error but do not fail job (table exists in S3)

• If Hudi properties are incorrect, Athena queries may fail

•

FR-CATALOG-004: Register Customer Aggregate Spend Table in Glue Data Catalog

• Title: Catalog Customer Aggregate Spend Analytics Table

• Description:

The system shall register the customeraggregatespend table in the AWS Glue Data Catalog to enable Athena querying for analytics.

• Business Objective:

Enable SQL-based analysis of customer spending patterns through Athena for business intelligence and reporting.

• Inputs:

• customeraggregatespend table at s3://adif-sdlc/analytics/customeraggregatespend/

• Processing Rules / Functional Steps:

1. Create or update Glue Catalog table with the following properties:

• Database: gen_ai_poc_databrickscoe

• Table name: customeraggregatespend

• Location: s3://adif-sdlc/analytics/customeraggregatespend/

• Schema: Name, TotalAmount, Date

2. Update table metadata and statistics

• Outputs:

• Glue Catalog table: gen_ai_poc_databrickscoe.customeraggregatespend

• Table queryable via Athena

• Preconditions:

• customeraggregatespend data has been written to S3 (FR-AGG-001)

• Glue database gen_ai_poc_databrickscoe exists

• Postconditions:

• customeraggregatespend table is queryable via Athena

• Table metadata is current and accurate

• Error / Validation Conditions:

• If S3 data does not exist, job must fail

• If catalog registration fails, log error but do not fail job (table exists in S3)

•

3. Non-Functional Considerations

Monitoring and Observability

• All AWS Glue jobs must log execution metrics to CloudWatch

• Job success/failure status must be captured

• Record counts at each processing stage should be logged

• Processing duration should be tracked

Data Governance (Optional)

• AWS Lake Formation may be used for fine-grained access control

• Data catalog permissions should be managed through Lake Formation policies if enabled

• Column-level security can be applied to sensitive customer information

Performance

• Processing should be optimized for the volume of data in source S3 paths

• Hudi table compaction should be considered for long-term performance

• Partitioning strategy for large datasets should be evaluated in technical design

Scalability

• Solution should handle growth in customer and order data volumes

• Glue job DPU allocation should be configurable based on data volume

•

4. Requirement Traceability Notes

Requirement ID	BRD Section	Description
---	---	---
FR-INGEST-001	Source Input (AWS S3) - Customer CSV	Customer data ingestion from s3://adif-sdlc/sdlc_wizard/customerdata/
FR-INGEST-002	Source Input (AWS S3) - Order CSV	Order data ingestion from s3://adif-sdlc/sdlc_wizard/orderdata/
FR-CLEAN-001	Data Cleaning - Remove "Null" and NULL	Null value removal from both datasets
FR-CLEAN-002	Data Cleaning - Remove duplicates	Duplicate removal from both datasets
FR-CATALOG-001	Glue Catalog Tables - sdlc_wizard_customer	Customer table registration in gen_ai_poc_databrickscoe
FR-CATALOG-002	Glue Catalog Tables - sdlc_wizard_order	Order table registration in gen_ai_poc_databrickscoe
FR-SCD2-001	SCD Type 2 (Hudi on S3)	SCD2 implementation for ordersummary with join logic
FR-INCR-001	Incremental Logic	Customer update-driven SCD2 processing
FR-AGG-001	Aggregation Table	Customer aggregate spend calculation and table creation
FR-CATALOG-003	Glue Catalog Tables - ordersummary	Order summary Hudi table registration
FR-CATALOG-004	Glue Catalog Tables - customeraggregatespend	Customer aggregate spend table registration

• Key Traceability Notes:

• All source paths are preserved exactly as specified in BRD

• Schema fields are documented as provided; datatypes are not assumed

• SCD2 logic follows BRD specification for join, insert, and close operations

• All S3 locations, database names, and table names match BRD exactly

• AWS service usage aligns with Architecture Components section of BRD